

MANUAL DE INCURSIÓN AL BACKEND

Protocolo de Reincorporación Técnica - Ingeniería de Software

Capítulo 1: El Cerebro en la Sombra

El Backend es la infraestructura lógica que sostiene la realidad de una aplicación. Si el Frontend es la interfaz (el hardware visual), el Backend es el kernel, el sistema que gestiona la persistencia, la seguridad y la orquestación de datos.

¿Por qué Backend?

Un cliente (navegador) es volátil. No puedes confiarle secretos, ni bases de datos, ni procesos pesados. El Backend existe para centralizar la verdad y aplicar reglas de negocio infranqueables antes de que un bit llegue al disco.

Capítulo 2: El Motor de Ejecución - Node.js

Node.js no es un lenguaje, es un entorno de ejecución. Utiliza el motor V8 de Chrome para compilar JavaScript directamente a lenguaje de máquina.

Concepto Crítico: No Bloqueante (Event Loop)

A diferencia de motores antiguos, Node no se detiene a esperar que un archivo se lea. Inicia la tarea, deja un puntero (Callback/Promesa) y sigue procesando. Esto permite que un solo hilo maneje miles de conexiones concurrentes sin colapsar.

¿Cómo usarlo? Ejecutas scripts desde la terminal con `node archivo.js`. Pero en desarrollo, usamos banderas como `--watch` para que el proceso se reinicie solo al detectar cambios en el código.

Capítulo 3: El Arsenal - NPM y Módulos

Tu sistema no sobrevive escribiendo todo desde cero. Usamos `npm` para gestionar dependencias externas.

- `npm init -y`: Inicializa el proyecto y crea el `package.json` (el manifiesto de tu sistema).
- `require()` / `import`: Es la forma en que el hardware lógico carga piezas de código aisladas (módulos).

Capítulo 4: La Interfaz de Red - Express.js

Express es el framework que simplifica el protocolo HTTP. Transforma los sockets crudos en una estructura de rutas manejable.

El Ciclo Req/Res

Request (req)	Response (res)
Datos que entran: <code>params</code> (URL), <code>query</code> (filtros), <code>body</code> (JSON).	Control de salida: <code>res.status()</code> , <code>res.json()</code> , <code>res.send()</code> .

¿Por qué Express? Porque gestionar cabeceras HTTP, códigos de estado y ruteo manualmente en Node puro es ineficiente y propenso a errores de seguridad.

Capítulo 5: Ingeniería de Flujo - Middlewares y next()

Un servidor es una tubería. Los Middlewares son filtros en esa tubería. La función `next()` es el comando que permite que el paquete de datos avance al siguiente filtro.

Error de Junior: Si no llamas a `next()` y no envías una respuesta, el socket queda colgado. El servidor quema recursos esperando y el cliente recibe un Timeout.

Capítulo 6: Arquitectura de Élite - MVC

No escribas "código espagueti". Separamos responsabilidades para mantener la integridad:

- **Router:** Solo dirige el tráfico.
- **Controller:** Recibe el Request, extrae datos, llama a la lógica y responde.
- **Service:** Donde vive la matemática pesada y la lógica de negocio pura.

Capítulo 7: Memoria Persistente (JSON vs DB)

Para tu TP3, usarás archivos `.json`. Esto simula una base de datos mediante el módulo `fs`. En sistemas de producción, escalamos a un ORM como **Sequelize** para mapear objetos a tablas SQL reales.

```
// Ejemplo de persistencia en JSON
const data = fs.readFileSync('./db.json', 'utf-8');
const db = JSON.parse(data);
```

```
db.usuarios.push(nuevoUsuario);  
fs.writeFileSync('./db.json', JSON.stringify(db));
```

Capítulo 8: Control de Calidad - ESLint

El código sucio es código vulnerable. ESLint es un analizador estático que audita tu sintaxis en tiempo real. Configura reglas como `"no-console": "warn"` para evitar fugas de información en logs de producción.

Capítulo 9: Despliegue en Render

El código no sirve si solo corre en tu máquina. Render es el entorno donde tu backend se vuelve público. Debes configurar variables de entorno (`process.env.PORT`) porque no sabes en qué puerto el hardware de Render va a inyectar tu aplicación.

Final del Protocolo. Prepárate para la ejecución del TP3.